HTTP QUERY Method:
A Security-Focused Literature Survey and Analysis

by

Anish Shobith

Registration Number

M.E. CyberSecurity

Under the guidance of

Panel Member 1:

Name

Designation

Manipal School of Information Sciences

MAHE, Manipal

Panel Member 2:

Name

Designation

Manipal School of Information Sciences

MAHE, Manipal

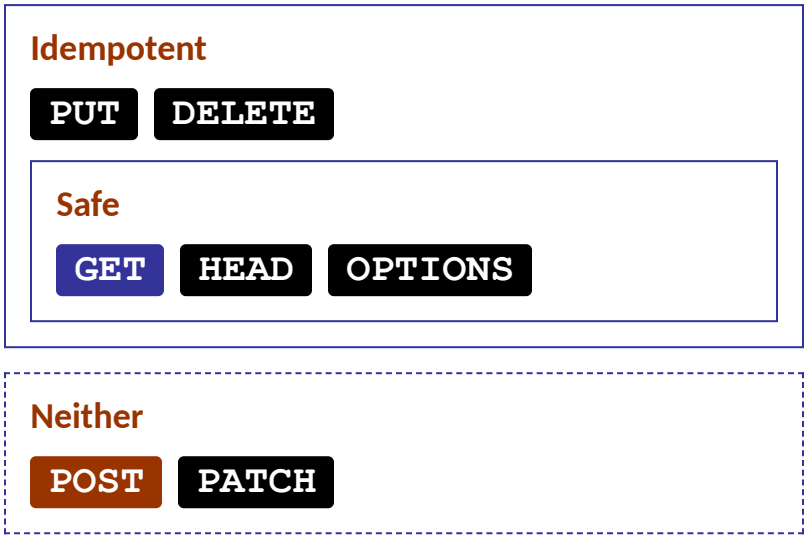
Presentation Outline

1. Introduction: HTTP methods, URIs and the GET / POST semantic gap
2. Objectives
3. Literature Review: standards and peer-reviewed studies
4. Description: the QUERY method (RFC 10008) and its security analysis
5. Conclusion

References

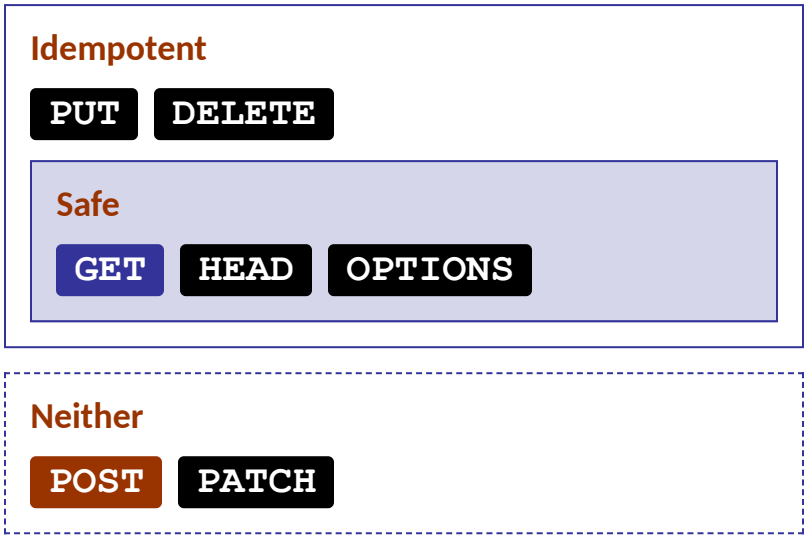
Why do HTTP request methods exist?

The method is the primary source of request semantics: it tells **every hop** what the client intends. [RFC 9110 §9.1]



Why do HTTP request methods exist?

The method is the primary source of request semantics: it tells **every hop** what the client intends. [RFC 9110 §9.1]

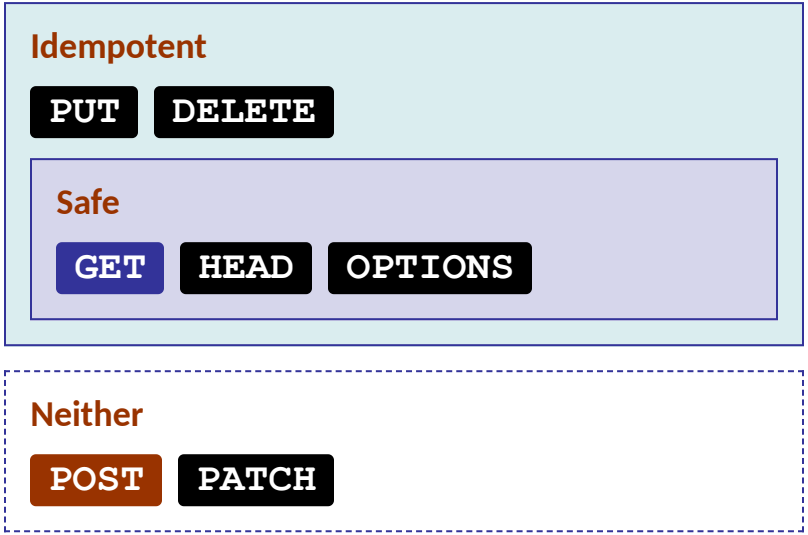


- **Safe**: no state change is requested, so caches, prefetchers and crawlers can send it freely

  [RFC 9110 §9.2.1]

Why do HTTP request methods exist?

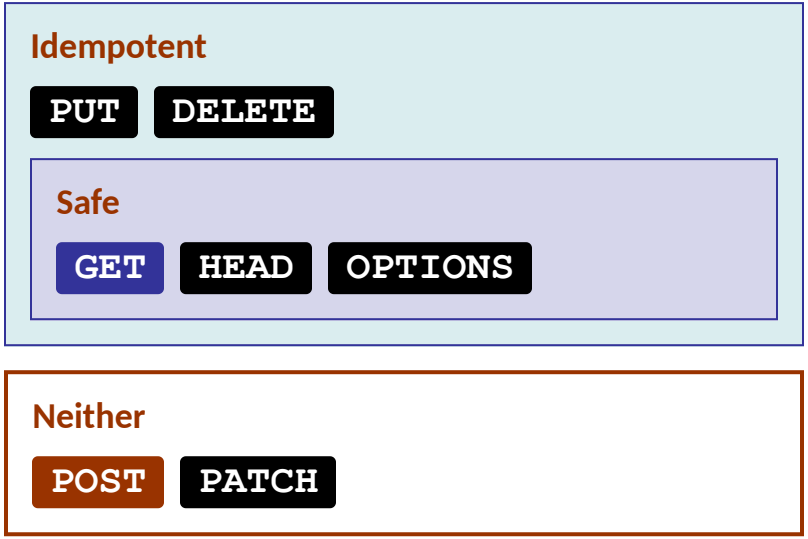
The method is the primary source of request semantics: it tells **every hop** what the client intends. [RFC 9110 §9.1]



- **Safe**: no state change is requested, so caches, prefetchers and crawlers can send it freely [RFC 9110 §9.2.1]
- **Idempotent**: repeating it has the same intended effect, so a client may retry after a lost connection [RFC 9110 §9.2.2]

Why do HTTP request methods exist?

The method is the primary source of request semantics: it tells **every hop** what the client intends. [RFC 9110 §9.1]



- **Safe**: no state change is requested, so caches, prefetchers and crawlers can send it freely [RFC 9110 §9.2.1]
- **Idempotent**: repeating it has the same intended effect, so a client may retry after a lost connection [RFC 9110 §9.2.2]
- **Neither**: no blind retries; a proxy **MUST NOT** retry these automatically [RFC 9110 §9.2.2]

Anatomy of an HTTP exchange

Request

```
GET 1 /products?category=laptop 2 HTTP/1.1
Host: example.com 3
Accept: application/json
Authorization: Bearer eyJhbGci…

(no content) 4
```

Response

```
HTTP/1.1 200 OK 5
Content-Type: application/json

{ 6
 "results": [ … ]
}
```

Anatomy of an HTTP exchange

Request

```
GET (1) /products?category=laptop (2) HTTP/1.1
Host: example.com (3)
Accept: application/json
Authorization: Bearer eyJhbGci…

(no content) (4)
```

Response

```
HTTP/1.1 200 OK (5)
Content-Type: application/json

{ (6)
 "results": [ … ]
}
```

(1) **Method**: what the client intends

Anatomy of an HTTP exchange

Request

```
GET (1) /products?category=laptop (2) HTTP/1.1
Host: example.com (3)
Accept: application/json
Authorization: Bearer eyJhbGci…

(no content) (4)
```

Response

```
HTTP/1.1 200 OK (5)
Content-Type: application/json

{ (6)
 "results": [ … ]
}
```

1. **Method**: what the client intends

2. **Target**: URI of the resource, including its query

Anatomy of an HTTP exchange

Request

```
GET (1) /products?category=laptop (2) HTTP/1.1
Host: example.com (3)
Accept: application/json
Authorization: Bearer eyJhbGci…

(no content) (4)
```

Response

```
HTTP/1.1 200 OK (5)
Content-Type: application/json

{ (6)
 "results": [ … ]
}
```

1. **Method**: what the client intends

2. **Target**: URI of the resource, including its query
3. **Headers**: metadata, credentials, formats

Anatomy of an HTTP exchange

Request

```
GET (1) /products?category=laptop (2) HTTP/1.1
Host: example.com (3)
Accept: application/json
Authorization: Bearer eyJhbGci…

(no content) (4)
```

Response

```
HTTP/1.1 200 OK (5)
Content-Type: application/json

{ (6)
 "results": [ … ]
}
```

1. **Method**: what the client intends
2. **Target**: URI of the resource, including its query
3. **Headers**: metadata, credentials, formats
4. **Content (body)**: none; GET content has no defined semantics [RFC 9110 §9.3.1]

Anatomy of an HTTP exchange

Request

```
GET (1) /products?category=laptop (2) HTTP/1.1
Host: example.com (3)
Accept: application/json
Authorization: Bearer eyJhbGci…

(no content) (4)
```

Response

```
HTTP/1.1 200 OK (5)
Content-Type: application/json

{ (6)
 "results": [ … ]
}
```

1. **Method**: what the client intends
2. **Target**: URI of the resource, including its query
3. **Headers**: metadata, credentials, formats
4. **Content (body)**: none; GET content has no defined semantics [RFC 9110 §9.3.1]
5. **Status line**: the outcome, as a code

Anatomy of an HTTP exchange

Request

```
GET (1) /products?category=laptop (2) HTTP/1.1
Host: example.com (3)
Accept: application/json
Authorization: Bearer eyJhbGci…

(no content) (4)
```

Response

```
HTTP/1.1 200 OK (5)
Content-Type: application/json

{ (6)
 "results": [ … ]
}
```

1. **Method**: what the client intends
2. **Target**: URI of the resource, including its query
3. **Headers**: metadata, credentials, formats
4. **Content (body)**: none; GET content has no defined semantics [RFC 9110 §9.3.1]
5. **Status line**: the outcome, as a code
6. **Response headers + content**: the result

Anatomy of an HTTP exchange

Request

```
GET (1) /products?category=laptop (2) HTTP/1.1
Host: example.com (3)
Accept: application/json
Authorization: Bearer eyJhbGci…

(no content) (4)
```

Response

```
HTTP/1.1 200 OK (5)
Content-Type: application/json

{ (6)
 "results": [ … ]
}
```

1. **Method**: what the client intends
2. **Target**: URI of the resource, including its query
3. **Headers**: metadata, credentials, formats
4. **Content (body)**: none; GET content has no defined semantics [RFC 9110 §9.3.1]
5. **Status line**: the outcome, as a code
6. **Response headers + content**: the result

Anatomy of a URI

```
https://example.com:443/search?q=security&page=2#results
```

Anatomy of a URI

`https://example.com:443/search?q=security&page=2#results`

scheme | authority | path

Anatomy of a URI

`https://example.com:443/search?q=security&page=2#results`

scheme | authority | path | query

- **query**: non-hierarchical data that, with the path, **identifies** a resource [RFC 3986 §3.4]

Anatomy of a URI



- **query**: non-hierarchical data that, with the path, **identifies** a resource [RFC 3986 §3.4]
- **fragment**: handled by the user agent, **never sent** in the HTTP request [RFC 3986 §3.5]

Anatomy of a URI

`https://example.com:443/search?q=security&page=2#results`

scheme | authority | path | query | fragment

- **query**: non-hierarchical data that, with the path, **identifies** a resource [RFC 3986 §3.4]
- **fragment**: handled by the user agent, **never sent** in the HTTP request [RFC 3986 §3.5]

URI query component

`?q=security&page=2`

Part of the resource **identifier** [RFC 3986 §3.4]

≠

HTTP QUERY method

`QUERY` `/search + content`

A request **method**; the query input travels as request content [RFC 10008 §1.1]

When the URI becomes the problem

Query parameters suit small inputs: **filter · search · sort · paginate · select**

```
GET /search?q=laptop
```

When the URI becomes the problem

Query parameters suit small inputs: **filter · search · sort · paginate · select**

```
GET /search?q=laptop&category=electronics&brand=lenovo,dell,hp
```

When the URI becomes the problem

Query parameters suit small inputs: **filter · search · sort · paginate · select**

```
GET /search?q=laptop&category=electronics&brand=lenovo,dell,hp&price_min=40000&price_max=100000&ram=16,32
```

- **No universal limit**: HTTP only recommends supporting ≥ 8000 octets [RFC 9110 §4.1]

When the URI becomes the problem

Query parameters suit small inputs: **filter · search · sort · paginate · select**

```
GET /search?q=laptop&category=electronics&brand=lenovo,dell,hp&price_min=40000&price_max=100000&ram=16,32&location=12.97,77.59&radius=10km&sort=price&page=2
```

- **No universal limit**: HTTP only recommends supporting ≥ 8000 octets [RFC 9110 §4.1]
- **Unknown real limits** across uncoordinated hops; servers may answer `414 URI Too Long` [RFC 10008 §1] [RFC 9110 §15.5.15]

When the URI becomes the problem

Query parameters suit small inputs: **filter · search · sort · paginate · select**

```
GET /search?q=laptop&category=electronics&brand=lenovo,dell,hp&price_min=40000&price_max=100000&ram=16,32&location=12.97,77.59&radius=10km&sort=price&page=2&filters=%7B%22gpu%22%3A%5B%22rtx-4060%22%2C%22rx-7600%22%5D%7D
```

- **No universal limit**: HTTP only recommends supporting ≥ 8000 octets [RFC 9110 §4.1]
- **Unknown real limits** across uncoordinated hops; servers may answer `414 URI Too Long` [RFC 10008 §1] [RFC 9110 §15.5.15]
- **Encoding overhead**: structured or binary data must be percent-encoded [RFC 9205 §4.5.1]

When the URI becomes the problem

Query parameters suit small inputs: **filter · search · sort · paginate · select**

```
GET /search?q=laptop&category=electronics&brand=lenovo,dell,hp&price_min=40000&price_max=100000&ram=16,32&location=12.97,77.59&radius=10km&sort=price&page=2&filters=%7B%22gpu%22%3A%5B%22rtx-4060%22%2C%22rx-7600%22%5D%7D
```

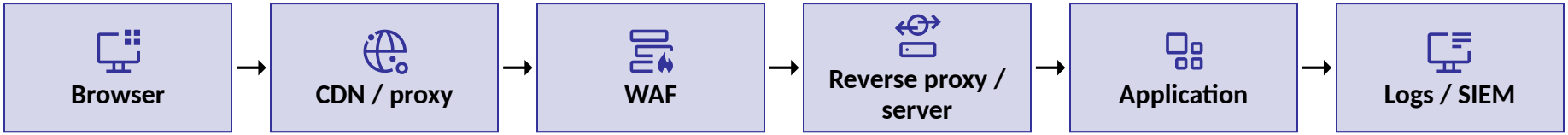
- **No universal limit**: HTTP only recommends supporting ≥ 8000 octets [RFC 9110 §4.1]
- **Unknown real limits** across uncoordinated hops; servers may answer `414 URI Too Long` [RFC 10008 §1] [RFC 9110 §15.5.15]
- **Encoding overhead**: structured or binary data must be percent-encoded [RFC 9205 §4.5.1]
- **More visible**: URIs are more likely than content to be logged or bookmarked [RFC 10008 §1]

→ as structured request content

```
{ "q": "laptop",
  "brand": ["lenovo","dell","hp"],
  "price": {"min":40000,"max":100000},
  "gpu": ["rtx-4060","rx-7600"] }
```

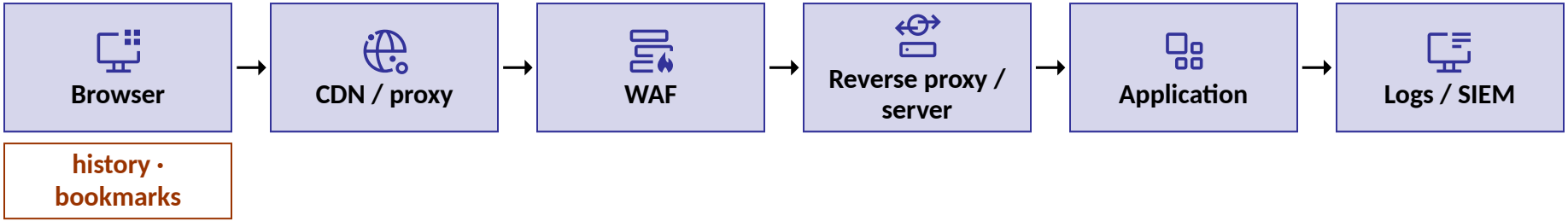
Where can URI query data be recorded?

```
GET /users/search?email=alice@example.com&role=admin HTTP/1.1
```



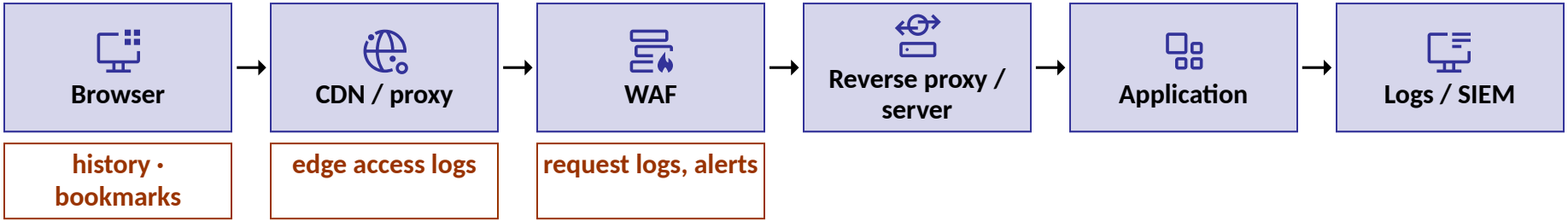
Where can URI query data be recorded?

```
GET /users/search?email=alice@example.com&role=admin HTTP/1.1
```



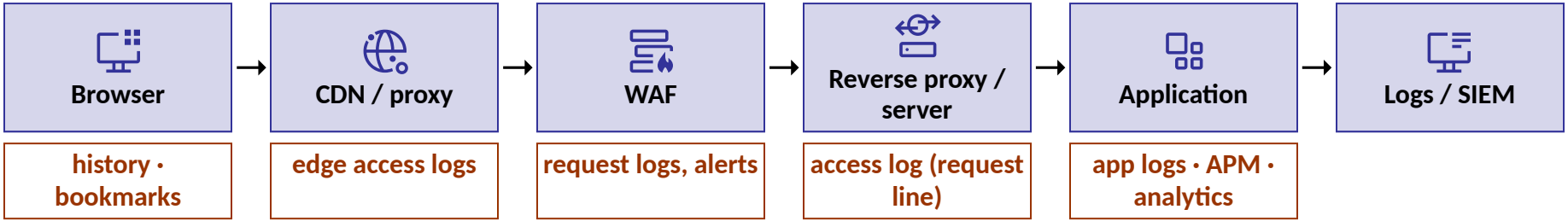
Where can URI query data be recorded?

```
GET /users/search?email=alice@example.com&role=admin HTTP/1.1
```



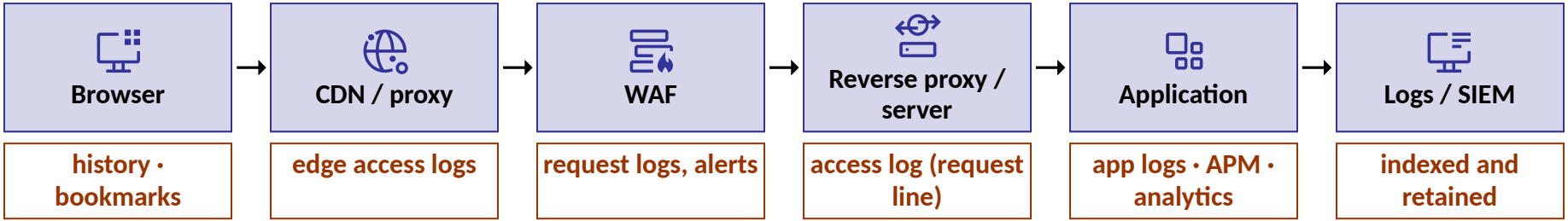
Where can URI query data be recorded?

```
GET /users/search?email=alice@example.com&role=admin HTTP/1.1
```



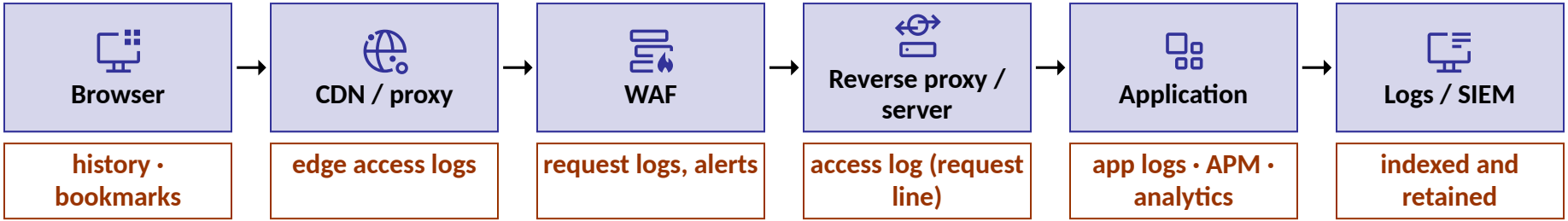
Where can URI query data be recorded?

```
GET /users/search?email=alice@example.com&role=admin HTTP/1.1
```



Where can URI query data be recorded?

`GET /users/search?email=alice@example.com&role=admin HTTP/1.1`



URI exposure surface

request line in access logs · history and bookmarks · `Referer` (same-origin, or a relaxed Referrer-Policy)

[RFC 9110 §17.9] [W3C Referrer Policy]

Request-content exposure surface

body logging and debug capture · WAF / APM inspection · any TLS-terminating intermediary. Less likely to be logged, *not never* [RFC 10008 §4]

Open issue: the semantic gap between GET and POST

	Query input in the **URI**	Query input in the **content**
Safe + idempotent	**GET** Retrieval; the query is part of the identifier	**?** **no standard method**
Unsafe / general processing	`GET /page?do=delete` Anti-pattern: owner MUST disallow unsafe actions via safe methods [RFC 9110 §9.2.1]	**POST** Resource-specific processing of the content [RFC 9110 §9.3.3]

Using POST for a query "hinders caching and uses an unsafe method for what would otherwise be a safe request." [RFC 9110 §17.9]

Open issue: the semantic gap between GET and POST

	Query input in the **URI**	Query input in the **content**
Safe + idempotent	**GET** Retrieval; the query is part of the identifier	**QUERY** Safe, idempotent query carried as content [RFC 10008 §2]
Unsafe / general processing	`GET /page?do=delete` Anti-pattern: owner MUST disallow unsafe actions via safe methods [RFC 9110 §9.2.1]	**POST** Resource-specific processing of the content [RFC 9110 §9.3.3]

Using POST for a query "hinders caching and uses an unsafe method for what would otherwise be a safe request." [RFC 9110 §17.9]

Open issue: the semantic gap between GET and POST

	Query input in the **URI**	Query input in the **content**
Safe + idempotent	**GET** Retrieval; the query is part of the identifier	**QUERY** Safe, idempotent query carried as content [RFC 10008 §2]
Unsafe / general processing	`GET /page?do=delete` Anti-pattern: owner MUST disallow unsafe actions via safe methods [RFC 9110 §9.2.1]	**POST** Resource-specific processing of the content [RFC 9110 §9.3.3]

Using POST for a query "hinders caching and uses an unsafe method for what would otherwise be a safe request." [RFC 9110 §17.9]

QUERY is **not a replacement** for GET or POST. It fills the gap between them.

2. Objectives

- Explain why HTTP distinguishes safe, idempotent and general-purpose methods
- Examine how query input in the URI creates size and exposure problems
- Analyse the HTTP QUERY method as standardised in RFC 10008
- Compare GET, POST and QUERY: semantics, caching, discovery and deployment
- Identify security implications for browsers, caches, WAFs, gateways and applications
- Survey the standards and peer-reviewed literature on the underlying mechanisms
- Recommend when to use GET, QUERY or POST

Authors and Title	Dataset	Purpose of Study	Key Findings
J. Reschke, J. M. Snell, M. Bishop, "The HTTP QUERY Method," RFC 10008, 2026	IETF Standards Track specification	Define a safe, idempotent method whose query is carried in the request content	Content-Type required; cacheable with content in the cache key; Accept-Query; Location / Content-Location; CORS preflight required
R. Fielding, M. Nottingham, J. Reschke (Eds.), "HTTP Semantics," RFC 9110, 2022	IETF Internet Standard (STD 97)	Define HTTP method semantics, safety and idempotency	Safe and idempotent describe client intent; URIs are not secured; POST for safe queries "uses an unsafe method" (§17.9)
M. Nottingham, "Building Protocols with HTTP," RFC 9205 (BCP 56), 2022	IETF Best Current Practice	Guide the design of HTTP-based APIs	GET queries can be unwieldy (URI syntax, encoding); POST queries lose caching and linking
A. Backman, J. Richer, M. Sporny (Eds.), "HTTP Message Signatures," RFC 9421, 2024	IETF Standards Track specification	Sign HTTP message components for integrity and authenticity	@query covers only the URI query; content needs Content-Digest (RFC 9530); query / form parameter shadowing

Authors and Title	Dataset	Purpose of Study	Key Findings
O. Starov, P. Gill, N. Nikiforakis, "Are You Sure You Want to Contact Us? Quantifying the Leakage of PII via Website Contact Forms," PoPETs, 2016	Contact pages of the 100,000 most popular websites	Measure leakage of personally identifiable information submitted via contact forms	PII leaked to third parties via third-party submissions, third-party scripts and the Referer header
H. V. Nguyen, L. Lo Iacono, H. Federrath, "Your Cache Has Fallen: Cache-Poisoned Denial-of-Service Attack," ACM CCS, 2019	15 web caching solutions (proxy caches and CDNs)	Study attacks where an origin error is cached and served to other users	Introduced CPDoS; 1 proxy cache and 5 CDN services vulnerable; one request can disable a site for a region
S. A. Mirheidari et al., "Cached and Confused: Web Cache Deception in the Wild," USENIX Security, 2020	340 high-profile sites from the Alexa Top 5K	Quantify web cache deception and path-confusion variants	Private data and auth tokens leaked via caches; many sites still vulnerable two years after disclosure

No peer-reviewed study of RFC 10008 itself was found (Crossref and web search, Sept 2026); these studies cover the mechanisms QUERY relies on.

Authors and Title	Dataset	Purpose of Study	Key Findings
B. Jabiyev, S. Sprecher, K. Onarlioglu, E. Kirda, "T-Reqs: HTTP Request Smuggling with Differential Fuzzing," ACM CCS, 2021	10 popular server, proxy and CDN technologies, tested in pairs	Systematically find HTTP processing discrepancies that enable request smuggling	Grammar-based differential fuzzing found new request manipulations and server pairs prone to smuggling
J. Chen et al., "We Still Don't Have Secure Cross-Domain Requests: an Empirical Study of CORS," USENIX Security, 2018	Real-world uses of CORS: design, implementation, deployment	Assess the security of CORS in practice	CORS relaxes cross-origin "write" in subtle ways, adds risky trust dependencies and is often misconfigured
A. Cha et al., "A Principled Approach to GraphQL Query Cost Analysis," ESEC/FSE, 2020	Query-response corpus for two commercial GraphQL APIs	Estimate query cost before execution	Linear-time static analysis gives tight upper bounds; existing analyses over- or under-estimate
O. Hartig, J. Pérez, "Semantics and Complexity of GraphQL," WWW, 2018	Formal model; experiments on practical implementations	Formalise GraphQL and analyse its complexity	Evaluation is NL-complete, yet responses can be prohibitively large; size is computable in polynomial time

Moving the query out of the URI: GET → POST → QUERY

```
GET /search?q=security&filters[year]=2026&filters[type]=rfc HTTP/1.1
Host: example.com
```

▲ query input sits in the URI

Safe
no state change requested

Idempotent
can be retried

Body-oriented
query is in the URI

GET Safe and idempotent, but the query is squeezed into the URI.

Moving the query out of the URI: GET → POST → QUERY

```
POST /search HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "q": "security",
  "filters": { "year": 2026, "type": "rfc" }
}
```

◀ same query input, now request content

Safe
not by definition

Idempotent
not by definition

Body-oriented
query is in the content

POST Query is in the content, but POST only says “process this”.

Moving the query out of the URI: GET → POST → QUERY

```
QUERY /search HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "q": "security",
  "filters": { "year": 2026, "type": "rfc" }
}
```

◀ same query input, now request content

Safe	Idempotent	Body-oriented
...	...	...

QUERY Query in the content **and** safe + idempotent semantics.

Moving the query out of the URI: GET → POST → QUERY

```
QUERY /search HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "q": "security",
  "filters": { "year": 2026, "type": "rfc" }
}
```

◀ same query input, now request content

Safe
per RFC 10008 §2

Idempotent
…

Body-oriented
…

QUERY Query in the content **and** safe + idempotent semantics.

Moving the query out of the URI: GET → POST → QUERY

```
QUERY /search HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "q": "security",
  "filters": { "year": 2026, "type": "rfc" }
}
```

◀ same query input, now request content

Safe
per RFC 10008 §2

Idempotent
per RFC 10008 §2

Body-oriented
...

QUERY Query in the content **and** safe + idempotent semantics.

Moving the query out of the URI: GET → POST → QUERY

```
QUERY /search HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "q": "security",
  "filters": { "year": 2026, "type": "rfc" }
}
```

◀ same query input, now request content

Safe
per RFC 10008 §2

Idempotent
per RFC 10008 §2

Body-oriented
query is in the content

QUERY Query in the content **and** safe + idempotent semantics.

Anatomy of a QUERY exchange (RFC 10008)

Request

```
QUERY 1 /feed 2 HTTP/1.1
Host: example.org
Content-Type: application/json 3
Accept: application/json

{ "q": "cybersecurity", 4
 "limit": 20, "sort": "relevance",
 "filters": { "year": 2026 } }
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Accept-Query: application/json 5
Location: /queries/42 6
Content-Location: /results/17 7

[ …results… ]
```

Anatomy of a QUERY exchange (RFC 10008)

Request

```
QUERY 1 /feed 2 HTTP/1.1
Host: example.org
Content-Type: application/json 3
Accept: application/json

{ "q": "cybersecurity", 4
 "limit": 20, "sort": "relevance",
 "filters": { "year": 2026 } }
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Accept-Query: application/json 5
Location: /queries/42 6
Content-Location: /results/17 7

[ …results… ]
```

1 **QUERY**: safe + idempotent [IANA registry]

Anatomy of a QUERY exchange (RFC 10008)

Request

```
QUERY 1 /feed 2 HTTP/1.1
Host: example.org
Content-Type: application/json 3
Accept: application/json

{ "q": "cybersecurity", 4
 "limit": 20, "sort": "relevance",
 "filters": { "year": 2026 } }
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Accept-Query: application/json 5
Location: /queries/42 6
Content-Location: /results/17 7

[ …results… ]
```

1. **QUERY**: safe + idempotent [IANA registry]
2. **Target**: the resource that scopes the query

Anatomy of a QUERY exchange (RFC 10008)

Request

```
QUERY (1) /feed (2) HTTP/1.1
Host: example.org
Content-Type: application/json (3)
Accept: application/json

{ "q": "cybersecurity", (4)
  "limit": 20, "sort": "relevance",
  "filters": { "year": 2026 } }
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Accept-Query: application/json (5)
Location: /queries/42 (6)
Content-Location: /results/17 (7)

[ …results… ]
```

1. **QUERY**: safe + idempotent [IANA registry]
2. **Target**: the resource that scopes the query
3. **Content-Type**: REQUIRED, never sniffed [RFC 10008 §2.1]

Anatomy of a QUERY exchange (RFC 10008)

Request

```
QUERY 1 /feed 2 HTTP/1.1
Host: example.org
Content-Type: application/json 3
Accept: application/json

{ "q": "cybersecurity", 4
 "limit": 20, "sort": "relevance",
 "filters": { "year": 2026 } }
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Accept-Query: application/json 5
Location: /queries/42 6
Content-Location: /results/17 7

[ …results… ]
```

1. **QUERY**: safe + idempotent [IANA registry]
2. **Target**: the resource that scopes the query
3. **Content-Type**: REQUIRED, never sniffed [RFC 10008 §2.1]

4. **Content**: the query itself (“query content”)

Anatomy of a QUERY exchange (RFC 10008)

Request

```
QUERY (1) /feed (2) HTTP/1.1
Host: example.org
Content-Type: application/json (3)
Accept: application/json

{ "q": "cybersecurity", (4)
 "limit": 20, "sort": "relevance",
 "filters": { "year": 2026 } }
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Accept-Query: application/json (5)
Location: /queries/42 (6)
Content-Location: /results/17 (7)

[ …results… ]
```

1. **QUERY**: safe + idempotent [IANA registry]
2. **Target**: the resource that scopes the query
3. **Content-Type**: REQUIRED, never sniffed [RFC 10008 §2.1]
4. **Content**: the query itself (“query content”)
5. **Accept-Query**: query formats accepted [RFC 10008 §3]

Anatomy of a QUERY exchange (RFC 10008)

Request

```
QUERY 1 /feed 2 HTTP/1.1
Host: example.org
Content-Type: application/json 3
Accept: application/json

{ "q": "cybersecurity", 4
 "limit": 20, "sort": "relevance",
 "filters": { "year": 2026 } }
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Accept-Query: application/json 5
Location: /queries/42 6
Content-Location: /results/17 7

[ …results… ]
```

1. **QUERY**: safe + idempotent [IANA registry]
2. **Target**: the resource that scopes the query
3. **Content-Type**: REQUIRED, never sniffed [RFC 10008 §2.1]
4. **Content**: the query itself (“query content”)
5. **Accept-Query**: query formats accepted [RFC 10008 §3]
6. **Location**: GET here repeats the query [RFC 10008 §2.4]
7. **Content-Location**: GET here fetches this result [RFC 10008 §2.3]

Anatomy of a QUERY exchange (RFC 10008)

Request

```
QUERY 1 /feed 2 HTTP/1.1
Host: example.org
Content-Type: application/json 3
Accept: application/json

{ "q": "cybersecurity", 4
 "limit": 20, "sort": "relevance",
 "filters": { "year": 2026 } }
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Accept-Query: application/json 5
Location: /queries/42 6
Content-Location: /results/17 7

[ …results… ]
```

1. **QUERY**: safe + idempotent [IANA registry]
2. **Target**: the resource that scopes the query
3. **Content-Type**: REQUIRED, never sniffed [RFC 10008 §2.1]
4. **Content**: the query itself (“query content”)
5. **Accept-Query**: query formats accepted [RFC 10008 §3]
6. **Location**: GET here repeats the query [RFC 10008 §2.4]
7. **Content-Location**: GET here fetches this result [RFC 10008 §2.3]

GET vs POST vs QUERY

Property	GET	POST	
Query input carried in	URI query component	Request content	
Safe	✓ Yes	✗ Not by definition	
Idempotent	✓ Yes	✗ Not by definition	
Large, structured input	Awkward: URI limits, encoding	✓ Yes	
Caching	Mature; key = method + URI	Rarely implemented	
URI for the query	Yes, by definition	No	
Ecosystem support	Mature	Mature	
Primary purpose	Retrieval	General processing	

Sources: [RFC 10008 §1] Table 1 · [RFC 9110 §9.2] · [RFC 9110 §9.2.3] · [RFC 9111 §2]

GET vs POST vs QUERY

Property	GET	POST	QUERY
Query input carried in	URI query component	Request content	Request content
Safe	✓ Yes	✗ Not by definition	✓ Yes
Idempotent	✓ Yes	✗ Not by definition	✓ Yes
Large, structured input	Awkward: URI limits, encoding	✓ Yes	✓ Yes
Caching	Mature; key = method + URI	Rarely implemented	Allowed; key MUST include content
URI for the query	Yes, by definition	No	Optional (Location)
Ecosystem support	Mature	Mature	Emerging (2026)
Primary purpose	Retrieval	General processing	Safe query with content

Sources: [RFC 10008 §1] Table 1 · [RFC 9110 §9.2] · [RFC 9110 §9.2.3] · [RFC 9111 §2]

Safe ≠ secure · Idempotent ≠ harmless

Safe
"the client does not request, and does not expect, any state change" [RFC 9110 §9.2.1]

Idempotent
several identical requests have the same *intended* effect as one [RFC 9110 §9.2.2]

Both describe the client's intent
RFC 9110: safety "does not prevent an implementation from including behavior that is potentially harmful".

A safe, idempotent QUERY can still enable:

Safe ≠ secure · Idempotent ≠ harmless

Safe
"the client does not request, and does not expect, any state change" [RFC 9110 §9.2.1]

Idempotent
several identical requests have the same *intended* effect as one [RFC 9110 §9.2.2]

Both describe the client's intent
RFC 9110: safety "does not prevent an implementation from including behavior that is potentially harmful".

A safe, idempotent QUERY can still enable:

- **Resource exhaustion**: huge, nested, high-cardinality queries [Cha et al., 2020]

Safe ≠ secure · Idempotent ≠ harmless

Safe
"the client does not request, and does not expect, any state change" [RFC 9110 §9.2.1]

Idempotent
several identical requests have the same *intended* effect as one [RFC 9110 §9.2.2]

Both describe the client's intent
RFC 9110: safety "does not prevent an implementation from including behavior that is potentially harmful".

A safe, idempotent QUERY can still enable:

- **Resource exhaustion**: huge, nested, high-cardinality queries [Cha et al., 2020]
- **Information disclosure**: results the caller should never see

Safe ≠ secure · Idempotent ≠ harmless

Safe
"the client does not request, and does not expect, any state change" [RFC 9110 §9.2.1]

Idempotent
several identical requests have the same *intended* effect as one [RFC 9110 §9.2.2]

Both describe the client's intent
RFC 9110: safety "does not prevent an implementation from including behavior that is potentially harmful".

A safe, idempotent QUERY can still enable:

- **Resource exhaustion**: huge, nested, high-cardinality queries [Cha et al., 2020]
- **Information disclosure**: results the caller should never see
- **Broken authorization**: read-only is not the same as allowed

Safe ≠ secure · Idempotent ≠ harmless

Safe
"the client does not request, and does not expect, any state change" [RFC 9110 §9.2.1]

Idempotent
several identical requests have the same *intended* effect as one [RFC 9110 §9.2.2]

Both describe the client's intent
RFC 9110: safety "does not prevent an implementation from including behavior that is potentially harmful".

A safe, idempotent QUERY can still enable:

- **Resource exhaustion**: huge, nested, high-cardinality queries [Cha et al., 2020]
- **Information disclosure**: results the caller should never see
- **Broken authorization**: read-only is not the same as allowed
- **Injection**: query content is untrusted input (SQL, NoSQL, LDAP, search DSLs)

Safe ≠ secure · Idempotent ≠ harmless

Safe
"the client does not request, and does not expect, any state change" [RFC 9110 §9.2.1]

Idempotent
several identical requests have the same *intended* effect as one [RFC 9110 §9.2.2]

Both describe the client's intent
RFC 9110: safety "does not prevent an implementation from including behavior that is potentially harmful".

A safe, idempotent QUERY can still enable:

- **Resource exhaustion**: huge, nested, high-cardinality queries [Cha et al., 2020]
- **Information disclosure**: results the caller should never see
- **Broken authorization**: read-only is not the same as allowed
- **Injection**: query content is untrusted input (SQL, NoSQL, LDAP, search DSLs)
- **Automated repetition**: retries are expected, and so is scraping

Caching QUERY: the key must include the content

RFC 10008: the cache key MUST incorporate the request content and related metadata [RFC 10008 §2.7]

Reality: many caches only cache GET, so they key on the URI alone [RFC 9111 §2]

Caching QUERY: the key must include the content

RFC 10008: the cache key MUST incorporate the request content and related metadata [RFC 10008 §2.7]

Reality: many caches only cache GET, so they key on the URI alone [RFC 9111 §2]

`QUERY /users/search`
`{"role":"admin"}` → `method + URI + content → K1` → **Response A (admins)**

`QUERY /users/search`
`{"role":"user"}` → `method + URI + content → K2` → **Response B (users)**

Caching QUERY: the key must include the content

RFC 10008: the cache key MUST incorporate the request content and related metadata [RFC 10008 §2.7]

Reality: many caches only cache GET, so they key on the URI alone [RFC 9111 §2]

`QUERY /users/search` `{"role":"admin"}` → `method + URI + content → K1` → **Response A (admins)**

`QUERY /users/search` `{"role":"user"}` → `method + URI + content → K2` → **Response B (users)**

`QUERY /users/search` `{"role":"user"}` → `content ignored, or role normalized away → K1` → **✗ Response A served**

Caching QUERY: the key must include the content

RFC 10008: the cache key MUST incorporate the request content and related metadata [RFC 10008 §2.7]

Reality: many caches only cache GET, so they key on the URI alone [RFC 9111 §2]

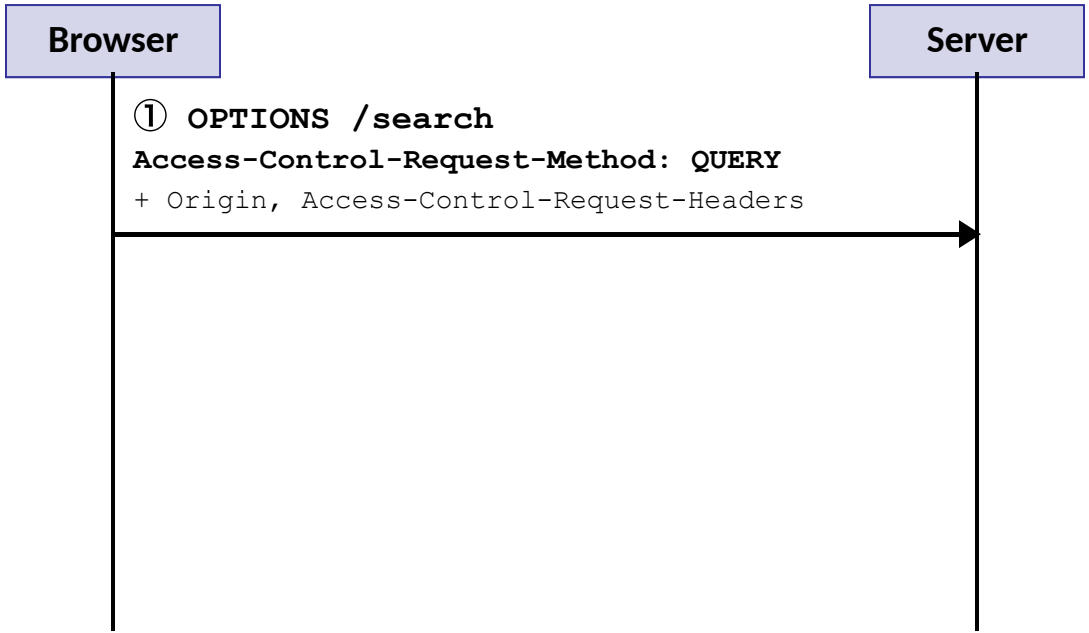
`QUERY /users/search` `{"role":"admin"}` → `method + URI + content → K1` → **Response A (admins)**

`QUERY /users/search` `{"role":"user"}` → `method + URI + content → K2` → **Response B (users)**

`QUERY /users/search` `{"role":"user"}` → `content ignored, or role normalized away → K1` → **✗ Response A served**

Normalization MAY drop insignificant differences (e.g. content encoding); a false positive returns the wrong response [RFC 10008 §4]. Cache ↔ origin disagreement drives cache poisoning and deception [Nguyen et al., 2019] [Mirheidari et al., 2020]

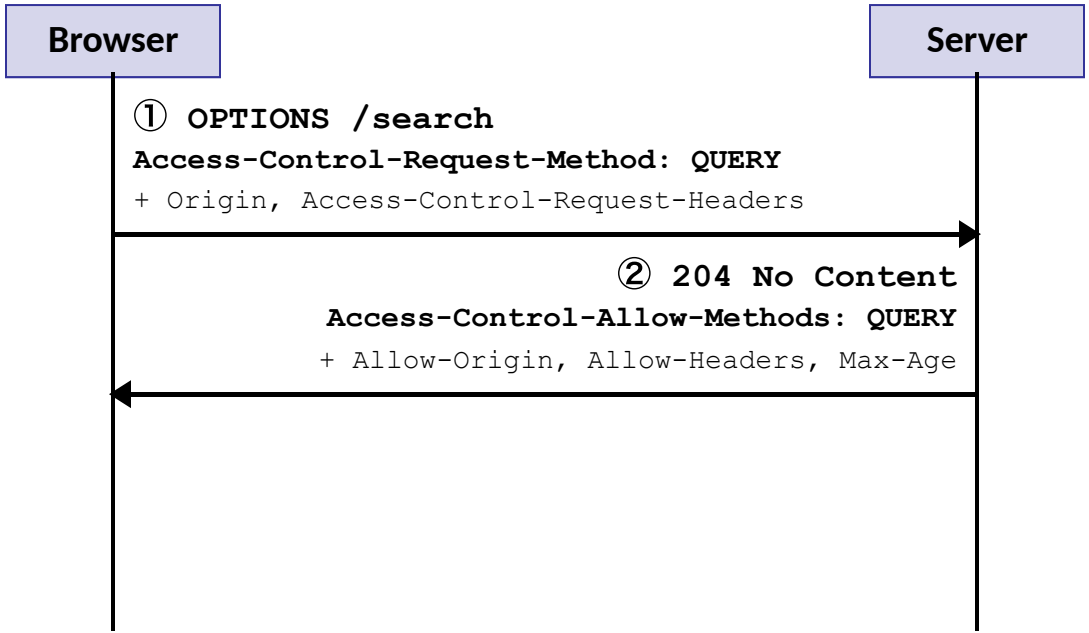
QUERY from the browser: CORS preflight



Is a preflight needed?

- **Yes**: cross-origin `fetch()`; only GET, HEAD, POST are CORS-safelisted [Fetch] [RFC 10008 §4]
- **No**: same-origin requests
- **No**: non-browser clients (curl, service-to-service)

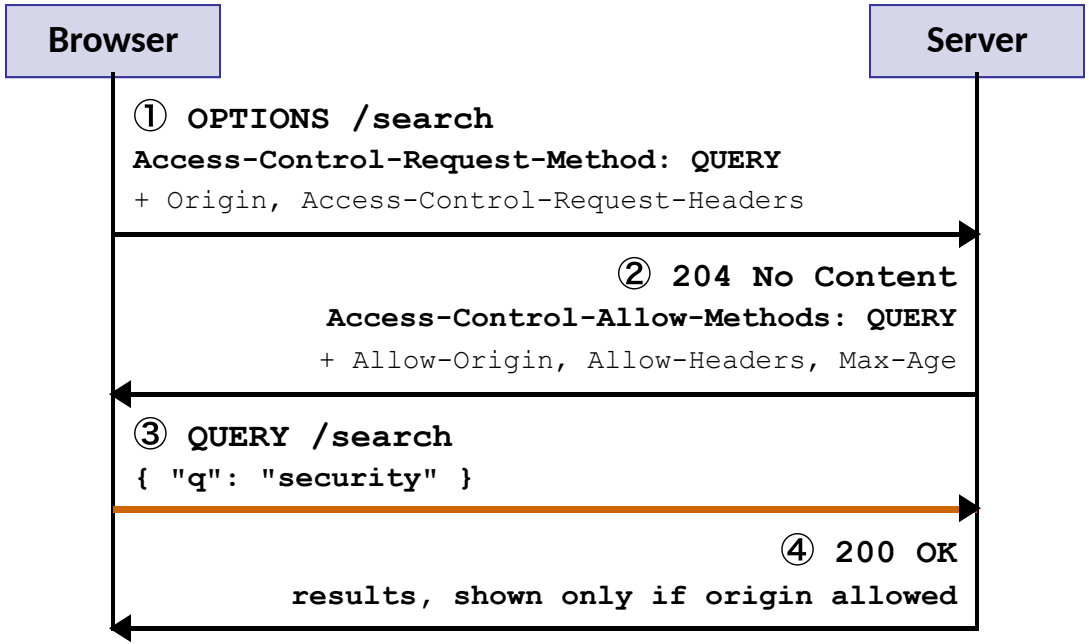
QUERY from the browser: CORS preflight



Is a preflight needed?

- **Yes**: cross-origin `fetch()`; only GET, HEAD, POST are CORS-safelisted [Fetch] [RFC 10008 §4]
- **No**: same-origin requests
- **No**: non-browser clients (curl, service-to-service)

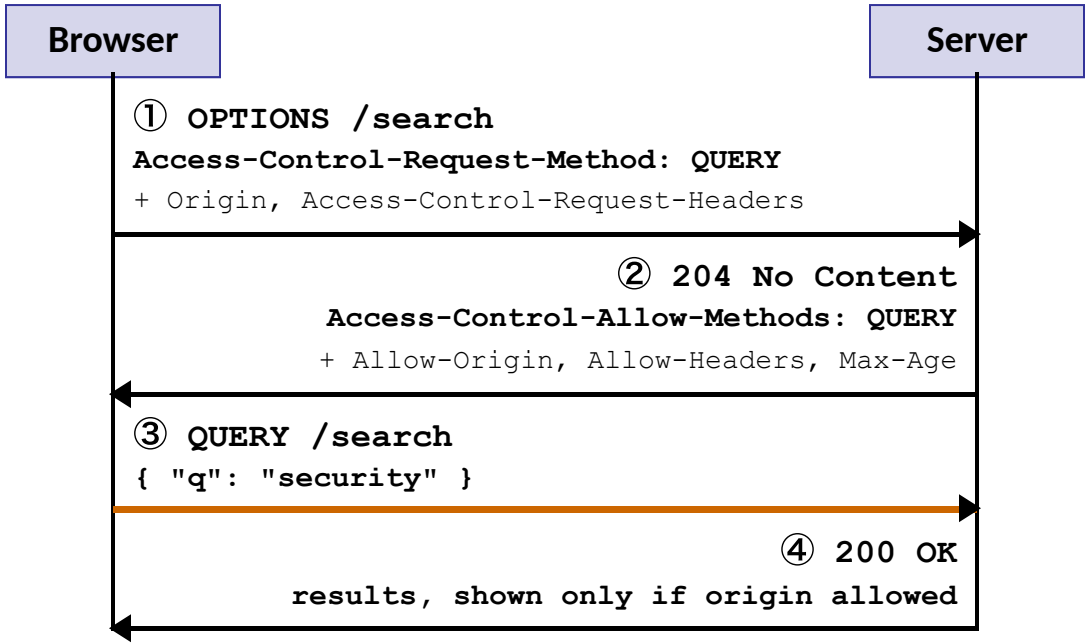
QUERY from the browser: CORS preflight



Is a preflight needed?

- **Yes**: cross-origin `fetch()`; only GET, HEAD, POST are CORS-safelisted [Fetch] [RFC 10008 §4]
- **No**: same-origin requests
- **No**: non-browser clients (curl, service-to-service)

QUERY from the browser: CORS preflight



Is a preflight needed?

- **Yes**: cross-origin `fetch()`; only GET, HEAD, POST are CORS-safelisted [Fetch] [RFC 10008 §4]
- **No**: same-origin requests
- **No**: non-browser clients (curl, service-to-service)

Forms send only GET / POST and `no-cors` rejects QUERY, so a cross-site page cannot send it **without server opt-in**. A permissive CORS policy can still leak results. [HTML] [Chen et al., 2018]

QUERY and HTTP Message Signatures (RFC 9421)

Signed component	What it covers	In a QUERY request
`@method`	request method	✓ QUERY
`@target-uri`	full target URI	✓ …/search
`@query`	URI query component	✗ usually just `?`
`@query-param`	one URI query parameter	✗ nothing in the content
`content-digest`	hash of the content [RFC 9530]	✓ the query itself

QUERY and HTTP Message Signatures (RFC 9421)

Signed component	What it covers	In a QUERY request
`@method`	request method	✓ QUERY
`@target-uri`	full target URI	✓ …/search
`@query`	URI query component	✗ usually just `?`
`@query-param`	one URI query parameter	✗ nothing in the content
`content-digest`	hash of the content [RFC 9530]	✓ the query itself

- **Sign the content**: cover `content-digest`; the verifier must recompute it [RFC 9421 §7.2.8]

QUERY and HTTP Message Signatures (RFC 9421)

Signed component	What it covers	In a QUERY request
`@method`	request method	✓ QUERY
`@target-uri`	full target URI	✓ …/search
`@query`	URI query component	✗ usually just `?`
`@query-param`	one URI query parameter	✗ nothing in the content
`content-digest`	hash of the content [RFC 9530]	✓ the query itself

- **Sign the content**: cover `content-digest`; the verifier must recompute it [RFC 9421 §7.2.8]
- **Parameter shadowing**: frameworks that merge URI and form parameters let an unsigned body value override a signed one [RFC 9421 §7.5.8]

QUERY and HTTP Message Signatures (RFC 9421)

Signed component	What it covers	In a QUERY request
`@method`	request method	✓ QUERY
`@target-uri`	full target URI	✓ …/search
`@query`	URI query component	✗ usually just `?`
`@query-param`	one URI query parameter	✗ nothing in the content
`content-digest`	hash of the content [RFC 9530]	✓ the query itself

- **Sign the content**: cover `content-digest`; the verifier must recompute it [RFC 9421 §7.2.8]
- **Parameter shadowing**: frameworks that merge URI and form parameters let an unsigned body value override a signed one [RFC 9421 §7.5.8]
- **Re-encoding** intermediaries change the digest and must re-sign [RFC 9421 §7.2.8]

QUERY and HTTP Message Signatures (RFC 9421)

Signed component	What it covers	In a QUERY request
`@method`	request method	✓ QUERY
`@target-uri`	full target URI	✓ …/search
`@query`	URI query component	✗ usually just `?`
`@query-param`	one URI query parameter	✗ nothing in the content
`content-digest`	hash of the content [RFC 9530]	✓ the query itself

- **Sign the content**: cover `content-digest`; the verifier must recompute it [RFC 9421 §7.2.8]
- **Parameter shadowing**: frameworks that merge URI and form parameters let an unsigned body value override a signed one [RFC 9421 §7.5.8]
- **Re-encoding** intermediaries change the digest and must re-sign [RFC 9421 §7.2.8]
- **Integrity and origin only**: not authorization, validation or confidentiality [RFC 9421 §1]

Threat model along the request path

- Browser / client
- CDN / cache
- WAF
- Gateway / proxy
- Application
- Database / search

Sources: [RFC 10008 §2.7] [Fetch] [RFC 9112 §6] [Jabiyev et al., 2021] [Chen et al., 2016] [OWASP WSTG-CONF-06]

Threat model along the request path

- **Browser / client**
- **CDN / cache**
- **WAF**
- **Gateway / proxy**
- **Application**
- **Database / search**

CORS preflight and allowed origins · credentials mode · exact method casing

Sources: [RFC 10008 §2.7] [Fetch] [RFC 9112 §6] [Jabiyev et al., 2021] [Chen et al., 2016] [OWASP WSTG-CONF-06]

Threat model along the request path

Browser / client	CORS preflight and allowed origins · credentials mode · exact method casing
CDN / cache	cache key includes content + type · safe normalization · poisoning, deception
WAF	
Gateway / proxy	
Application	
Database / search	

Sources: [RFC 10008 §2.7] [Fetch] [RFC 9112 §6] [Jabiyev et al., 2021] [Chen et al., 2016] [OWASP WSTG-CONF-06]

Threat model along the request path

Browser / client	CORS preflight and allowed origins · credentials mode · exact method casing
CDN / cache	cache key includes content + type · safe normalization · poisoning, deception
WAF	method allowlist · inspect QUERY bodies like POST bodies · size limits
Gateway / proxy	
Application	
Database / search	

Sources: [RFC 10008 §2.7] [Fetch] [RFC 9112 §6] [Jabiyev et al., 2021] [Chen et al., 2016] [OWASP WSTG-CONF-06]

Threat model along the request path

Browser / client	CORS preflight and allowed origins · credentials mode · exact method casing
CDN / cache	cache key includes content + type · safe normalization · poisoning, deception
WAF	method allowlist · inspect QUERY bodies like POST bodies · size limits
Gateway / proxy	consistent message framing (no smuggling) · rate limits · 405 / 501 handling
Application	
Database / search	

Sources: [RFC 10008 §2.7] [Fetch] [RFC 9112 §6] [Jabiyev et al., 2021] [Chen et al., 2016] [OWASP WSTG-CONF-06]

Threat model along the request path

Browser / client	CORS preflight and allowed origins · credentials mode · exact method casing
CDN / cache	cache key includes content + type · safe normalization · poisoning, deception
WAF	method allowlist · inspect QUERY bodies like POST bodies · size limits
Gateway / proxy	consistent message framing (no smuggling) · rate limits · 405 / 501 handling
Application	authN + per-query authZ · strict Content-Type · validation · cost limits · body logging
Database / search	

Sources: [RFC 10008 §2.7] [Fetch] [RFC 9112 §6] [Jabiyev et al., 2021] [Chen et al., 2016] [OWASP WSTG-CONF-06]

Threat model along the request path

Browser / client	CORS preflight and allowed origins · credentials mode · exact method casing
CDN / cache	cache key includes content + type · safe normalization · poisoning, deception
WAF	method allowlist · inspect QUERY bodies like POST bodies · size limits
Gateway / proxy	consistent message framing (no smuggling) · rate limits · 405 / 501 handling
Application	authN + per-query authZ · strict Content-Type · validation · cost limits · body logging
Database / search	parameterised queries (no injection) · timeouts · result-size caps

Sources: [RFC 10008 §2.7] [Fetch] [RFC 9112 §6] [Jabiyev et al., 2021] [Chen et al., 2016] [OWASP WSTG-CONF-06]

Worked example: a user-search API

Concern	GET `?email=alice@…`	POST `{"email": …}`	QUERY `{"email": …}`
Email in the URI (logs, history)	✗ Exposed	✓ Avoided	✓ Avoided
Safe intent visible to caches	✓ Yes	✗ Opaque	✓ Yes
Body may be logged or inspected	n/a	! Yes	! Yes

Worked example: a user-search API

Concern	GET `?email=alice@…`	POST `{"email": …}`	QUERY `{"email": …}`
Email in the URI (logs, history)	✗ Exposed	✓ Avoided	✓ Avoided
Safe intent visible to caches	✓ Yes	✗ Opaque	✓ Yes
Body may be logged or inspected	n/a	! Yes	! Yes
Authorization for admin lookups	**Required**	**Required**	**Required**
Validation, injection defences	**Required**	**Required**	**Required**
Rate and cost limits (DoS)	**Required**	**Required**	**Required**

Worked example: a user-search API

Concern	GET `?email=alice@…`	POST `{"email": …}`	QUERY `{"email": …}`
Email in the URI (logs, history)	✗ Exposed	✓ Avoided	✓ Avoided
Safe intent visible to caches	✓ Yes	✗ Opaque	✓ Yes
Body may be logged or inspected	n/a	! Yes	! Yes
Authorization for admin lookups	**Required**	**Required**	**Required**
Validation, injection defences	**Required**	**Required**	**Required**
Rate and cost limits (DoS)	**Required**	**Required**	**Required**

QUERY changes the **semantics** and **where the data travels**. It does not remove the need for authorization, validation, logging hygiene or limits.

Standardised is not the same as deployed

Protocol level: set by RFC 10008

- safe + idempotent semantics
- Content-Type required; 4xx rules
- cacheable; content in the cache key
- Accept-Query, Location, Content-Location
- redirect rules; CORS preflight

Deployment level: decided at every hop

- method allowlists (LB, WAF, gateway): 405 / 501 [RFC 9110 §9.1]
- proxy / CDN forwarding and caching of QUERY
- framework routing and body parsing
- CORS configuration · clients: `"query"` ≠ `"QUERY"`
- logs, SIEM parsers · API docs: OpenAPI 3.2 has `query` [OAS 3.2]

Standardised is not the same as deployed

Protocol level: set by RFC 10008

- safe + idempotent semantics
- Content-Type required; 4xx rules
- cacheable; content in the cache key
- Accept-Query, Location, Content-Location
- redirect rules; CORS preflight

Deployment level: decided at every hop

- method allowlists (LB, WAF, gateway): 405 / 501 [RFC 9110 §9.1]
- proxy / CDN forwarding and caching of QUERY
- framework routing and body parsing
- CORS configuration · clients: `"query"` ≠ `"QUERY"`
- logs, SIEM parsers · API docs: OpenAPI 3.2 has `query` [OAS 3.2]

Browser → **CDN** → **Load balancer** → **WAF** → **Reverse proxy** → **Framework** → **Application**

One unsupported hop can break QUERY, or let it slip past a control.

GET

URI-oriented retrieval

- query fits in the URI
- links and caching matter

semantic gap →

QUERY

Safe, body-oriented query

- the operation *is* a query
- large or structured input
- every hop supports it

broader operations →

POST

General processing

- operation is not safe
- changes state, submits

GET

URI-oriented retrieval

- query fits in the URI
- links and caching matter

semantic gap →

QUERY

Safe, body-oriented query

- the operation *is* a query
- large or structured input
- every hop supports it

broader operations →

POST

General processing

- operation is not safe
- changes state, submits

- **QUERY is not a replacement for GET or POST.** It fills a semantic gap: safe, idempotent queries with structured request content.

Conclusion

GET

URI-oriented retrieval

- query fits in the URI
- links and caching matter

semantic gap →

QUERY

Safe, body-oriented query

- the operation *is* a query
- large or structured input
- every hop supports it

broader operations →

POST

General processing

- operation is not safe
- changes state, submits

- **QUERY is not a replacement for GET or POST.** It fills a semantic gap: safe, idempotent queries with structured request content.
- Its cybersecurity value depends on how clients, intermediaries, caches, gateways, applications and logs **implement** it, not on the method alone.

References

[1] J. Reschke, J. M. Snell, and M. Bishop, “The HTTP QUERY Method,” RFC 10008, IETF, Jun. 2026, doi: 10.17487/RFC10008.

[2] R. Fielding, M. Nottingham, and J. Reschke, Eds., “HTTP Semantics,” STD 97, RFC 9110, Jun. 2022.

[3] R. Fielding, M. Nottingham, and J. Reschke, Eds., “HTTP Caching,” STD 98, RFC 9111, Jun. 2022.

[4] R. Fielding, M. Nottingham, and J. Reschke, Eds., “HTTP/1.1,” STD 99, RFC 9112, Jun. 2022.

[5] T. Berners-Lee, R. Fielding, and L. Masinter, “Uniform Resource Identifier (URI): Generic Syntax,” STD 66, RFC 3986, Jan. 2005.

[6] M. Nottingham, “Building Protocols with HTTP,” BCP 56, RFC 9205, Jun. 2022.

[7] A. Backman, J. Richer, and M. Sporny, Eds., “HTTP Message Signatures,” RFC 9421, Feb. 2024.

[8] R. Polli and L. Pardue, “Digest Fields,” RFC 9530, Feb. 2024.

[9] WHATWG, “Fetch Standard” and “HTML Standard,” Living Standards.

[10] W3C, “Referrer Policy.”

[11] IANA, “Hypertext Transfer Protocol (HTTP) Method Registry.”

[12] OpenAPI Initiative, “OpenAPI Specification v3.2.0,” Sep. 2025.

[13] OWASP, “WSTG-CONF-06: Test HTTP Methods,” Web Security Testing Guide.

[14] MDN Web Docs, “Cross-Origin Resource Sharing (CORS).”

References

[15] O. Starov, P. Gill, and N. Nikiforakis, “Are You Sure You Want to Contact Us? Quantifying the Leakage of PII via Website Contact Forms,” Proc. Privacy Enhancing Technologies, vol. 2016, no. 1, pp. 20–33, 2016, doi: 10.1515/popets-2015-0028.

[16] H. V. Nguyen, L. Lo Iacono, and H. Federrath, “Your Cache Has Fallen: Cache-Poisoned Denial-of-Service Attack,” in Proc. ACM CCS, 2019, pp. 1915–1936, doi: 10.1145/3319535.3354215.

[17] S. A. Mirheidari, S. Arshad, K. Onarlioglu, B. Crispo, E. Kirda, and W. Robertson, “Cached and Confused: Web Cache Deception in the Wild,” in Proc. 29th USENIX Security Symposium, 2020.

[18] B. Jabiyev, S. Sprecher, K. Onarlioglu, and E. Kirda, “T-Reqs: HTTP Request Smuggling with Differential Fuzzing,” in Proc. ACM CCS, 2021, pp. 1805–1820, doi: 10.1145/3460120.3485384.

[19] J. Chen, J. Jiang, H. Duan, T. Wan, S. Chen, V. Paxson, and M. Yang, “We Still Don’t Have Secure Cross-Domain Requests: an Empirical Study of CORS,” in Proc. 27th USENIX Security Symposium, 2018.

[20] A. Cha, E. Wittern, G. Baudart, J. C. Davis, L. Mandel, and J. A. Laredo, “A Principled Approach to GraphQL Query Cost Analysis,” in Proc. ESEC/FSE, 2020, pp. 257–268, doi: 10.1145/3368089.3409670.

[21] O. Hartig and J. Pérez, “Semantics and Complexity of GraphQL,” in Proc. The Web Conference (WWW), 2018, pp. 1155–1164, doi: 10.1145/3178876.3186014.

[22] J. Chen, J. Jiang, H. Duan, N. Weaver, T. Wan, and V. Paxson, “Host of Troubles: Multiple Host Ambiguities in HTTP Implementations,” in Proc. ACM CCS, 2016, pp. 1516–1527, doi: 10.1145/2976749.2978394.

Queries

&

Suggestions

THANK YOU...